

Visible Tests Are Not Oracles: Characterizing False Acceptance in LLM-Based Repository Repair

Anonymous Authors

Abstract—Repository-level LLM repair systems routinely stop when a candidate applies cleanly and visible tests pass. In issue-driven benchmarks, this stopping rule can leave an important question unresolved: several visible-passing patches may satisfy the same public tests while disagreeing on the behavior implied by the issue. PatchCourt audits this acceptance layer. It treats visible-passing patches as executable hypotheses, compares them with layered public probes, records behavioral-disagreement certificates, and freezes the certificate index before held-out labels are opened. Post-freeze calibration then measures how often these accepted-looking records fail held-out issue tests. In a 50-task pilot, 200 candidate slots produce 68 visible-passing candidates and 37 raw certificates; conservative filtering yields 30 task-level audit records, all executable under held-out calibration. Within this selected certificate-producing path, 17/30 probe-accepted records become false acceptances and 26/30 retain specification-debt signals. Stronger issue-code or full-package probes preserve pass/fail disagreement in 6/10 selected upgrades, with another 6/10 package-level certificates in a separate upgrade set. A local Qwen stress lane adds one issue-specific and four package-integrity disagreement cases. The artifact regenerates the headline tables and links each count to certificate records. The results argue for shifting visible tests from a final repair endpoint to a pre-acceptance review trigger.

Index Terms—repository-level repair, large language models, software testing, program repair, test oracles, empirical software engineering

I. INTRODUCTION

Visible tests are an attractive stopping rule. They are executable, cheap relative to full benchmark evaluation, and naturally available to repository-level repair agents during development. Modern repair systems can search files, edit code, run tests, and iterate until the visible command set passes. The practical temptation is clear: if the patch applies and the visible commands pass, accept it.

The difficulty is that visible success is a local observation over an incomplete behavioral surface. A visible-passing patch can satisfy the available tests while still violating the intended behavior of the public issue. Repository-level repair amplifies this ambiguity: the same issue can admit superficially plausible patches that modify different call paths, preserve different edge cases, or mask failures in different ways. Visible success therefore leaves a question that is easy to miss during repair: did the acceptance step actually collapse the candidate space to behaviorally compatible fixes?

Consider a Matplotlib issue involving bar plots whose inputs are all NaN. Several main-lane candidates reach the visible-pass pool. A full-package issue probe then separates the pool: three candidates preserve a single bar object with a NaN coordinate, while another exits through the Matplotlib axes

stack. No developer patch is needed to observe the split. The resulting certificate is deliberately modest: it records that visible tests admitted behaviorally incompatible hypotheses.

False acceptance names the downstream failure of this acceptance step. PatchCourt studies it after candidate patches exist and before held-out labels are read. Its output is executable evidence that a visible-pass pool still contains unresolved behavior and should be inspected, reranked, or held back.

The key constraint is the gold boundary. Before certificates are frozen, PatchCourt uses public issue text, public repository snapshots, public snippets or reproducer text, candidate diffs, visible commands, and probe outcomes. Developer patches, test patches, FAIL_TO_PASS labels, PASS_TO_PASS labels, and held-out outcomes enter only after the audit index is frozen. This separation lets us ask whether pre-freeze evidence anticipates post-freeze false acceptance.

The study uses a frozen 30-certificate audit set drawn from SWE-bench-style repository repair tasks after visible-pass and pre-freeze public-probe evidence are available. The estimand is therefore conditional on a certificate-producing path: among auditable tasks that already expose conservative pre-freeze behavioral disagreement, how often does visible/probe acceptance fail held-out calibration? Within that selected path, all 30 held-out calibrations execute, and 17 of 30 visible/probe-accepted cases fail held-out FAIL_TO_PASS tests. Stronger issue-code and full-package probes show that the signal extends beyond source-isolated templates for selected targets. The same evidence also fixes the interpretation: the paper measures a high-risk, auditable acceptance regime; raw disagreement scores remain under-calibrated; and the Qwen lane is used as a stress test of candidate-source compatibility.

The contributions are:

- a characterization of visible-test false acceptance within a selected certificate-producing path under frozen, held-out calibration;
- the PatchCourt protocol, which records executable disagreement evidence before evaluation labels are read;
- a layered probe evaluation that separates source-isolated templates, issue-code probes, full-package issue probes, package-integrity checks, and held-out calibration;
- a triage and candidate-source stress analysis, including a local Qwen lane with low issue-specific but nonzero disagreement yield.

II. BACKGROUND AND PROBLEM

A. Repository-Level Repair Evaluation

SWE-bench-style tasks ask a system to modify a real repository snapshot to address a public issue [1]. This setting matters because it moves beyond single-function toy problems: agents must reason over real packages, build systems, dependencies, and test commands. Agent systems such as SWE-agent and AutoCodeRover have improved the ability of language models to navigate repositories, edit files, and run tests [2], [3]. The same setting makes evaluation harder. A patch may pass commands available during generation while failing hidden or held-out issue tests. This is the repository-level form of a long-standing automated program repair concern: plausible patches can overfit incomplete tests [4]–[6].

B. The Acceptance Gap

We use *false acceptance* to denote a visible/probe-accepted case that later fails the post-freeze held-out FAIL_TO_PASS layer. False acceptance is not identical to behavioral disagreement. A disagreement certificate can be a warning even when the selected candidate later passes held-out labels. Conversely, a held-out failure can occur without a strong pre-freeze certificate. PatchCourt is designed around that distinction: certificates are risk evidence, and held-out calibration measures how often that risk corresponds to benchmark failure.

C. Developer Patch Boundary

Prior differential patch testing can compare generated patches with a human-written ground-truth patch. Recent SWE-bench correctness work and PatchDiff show that many benchmark-plausible patches behave differently from developer patches and that some counted solutions fail developer-written tests [7]. That evidence is useful for retrospective benchmark studies, but developer patches are not available at deployment time and can leak the answer into inference. PatchCourt deliberately avoids developer patches before freeze. It compares candidate behaviors against public probes and other candidate hypotheses, then uses developer/test labels only for post-freeze calibration.

D. Running Example

Figure 1 gives the compact quantitative shape behind the opening Matplotlib case. The probe is a public, issue-facing measurement of the reported edge case. When the same probe is run across the visible-pass candidate pool, three candidates keep one bar whose height remains NaN, while one candidate fails through the Matplotlib axes stack. This split is the certificate: a cheap, executable record that visible tests admitted incompatible hypotheses before any held-out label was read.

III. PATCHCOURT

A. Overview

PatchCourt is a measurement and audit protocol for visible-passing repository patches. Figure 3 shows the main flow with the counted objects at each stage: candidate sources produce

Matplotlib-24149 probe outcome

visible-pass candidate pool, n=4

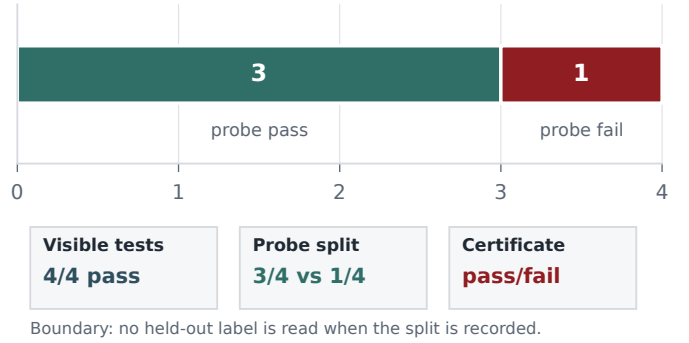


Fig. 1. Quantitative running example for a full-package public probe. All four visible-passing candidates pass visible tests; the public probe splits them into three probe-pass candidates and one probe-fail candidate before any held-out label is read.

```
[
  {
    "claim": "visible_pass_is_not_oracle",
    "task": "Matplotlib-24149", "case": "all-NaN bar",
    "edit": {
      "file": "lib/matplotlib/axes/_axes.py",
      "change": "guard all-NaN bar path"
    },
    "accepted_by": {
      "visible_tests": "pass"
    },
    "pre_freeze": {
      "probe": "full-package public probe",
      "split": "3 preserve NaN bar / 1 axes failure",
      "certificate": "pass_fail_bdc"
    },
    "audit": {
      "retain": true, "risk": "specification_debt"
    },
    "post_freeze": {
      "held_out": "fail",
      "outcome": "false_acceptance"
    },
    "review_action": "inspect_or_rerank_before_accept"
  }
]
```

Fig. 2. Core PatchCourt record for false acceptance. A visible-passing Matplotlib edit reaches the acceptance gate, a public full-package probe splits candidate behavior before freeze, audit retains the record as specification-debt evidence, and post-freeze calibration marks the accepted-looking case as false acceptance.

patches, visible tests define the accepted-looking pool, public probes measure behavior before freeze, the certificate index is audited and frozen, and held-out labels are read only afterward. The output is an evidence ledger whose rows bind a task, candidate source, probe layer, behavior signature, audit label, and calibration outcome. Figure 2 instantiates the record type behind the main result: visible-test pass is the acceptance gate, public-probe disagreement supplies pre-freeze evidence, and held-out calibration names the false-acceptance outcome.

B. The Gold Boundary Principle

Each task contains a public repository snapshot, public issue text, visible commands, and optional public snippets or reproducer fragments. Evaluation-only fields are excluded before freeze: developer patch, test patch, FAIL_TO_PASS labels, PASS_TO_PASS labels, and held-out outcomes. This boundary is the central design principle. The pre-freeze protocol may observe only information that an acceptance-time auditor could obtain without opening the benchmark answer key. The post-freeze phase then asks how often the frozen pre-freeze evidence coincides with held-out failure. Keeping those phases separate lets the paper study acceptance risk without converting public probes into hidden labels.

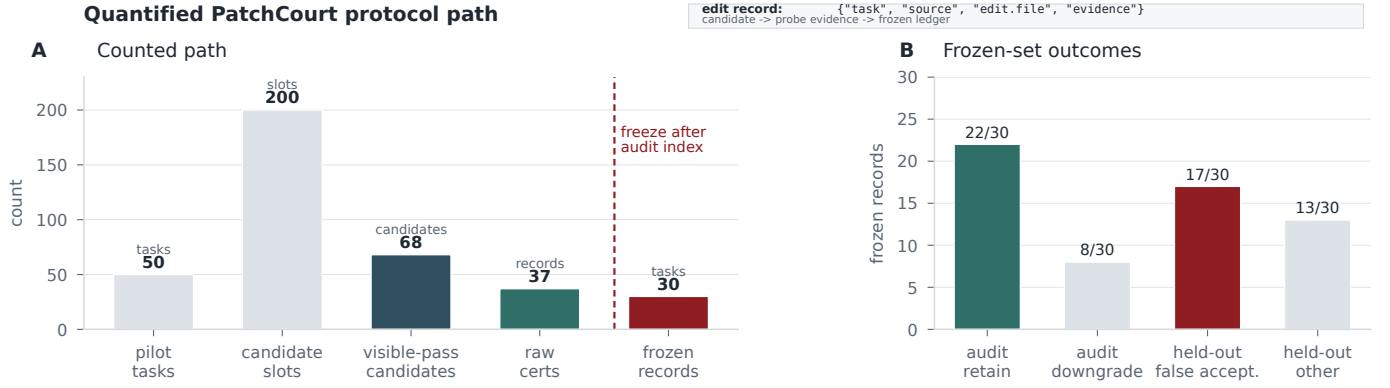


Fig. 3. Quantified PatchCourt protocol path with structured-edit strip. The left panel counts pilot tasks, candidate slots, visible-pass candidates, raw certificates, and frozen records; the right panel shows frozen-set audit and held-out outcome counts.

C. Inputs and Evidence Model

For task i , let R_i denote the repository snapshot, I_i the public issue context, V_i the visible command set, P_i the generated patch candidates, Q_i the public probe set, and H_i the held-out calibration layer. All pre-freeze evidence is a function of public inputs and candidate behavior:

$$\Omega_i^{\text{pre}} = R_i \cup I_i \cup V_i \cup P_i \cup Q_i, \quad \Omega_i^{\text{pre}} \cap H_i = \emptyset. \quad (1)$$

The freeze operator therefore selects records from pre-freeze evidence alone:

$$F(E^{\text{pre}}, H) = F(E^{\text{pre}}). \quad (2)$$

For a candidate patch $p \in P_i$, the visible-passing candidate set is

$$C_i = \{p \in P_i : \text{apply}_i(p) = 1 \wedge \text{visible}_i(p) = 1\}. \quad (3)$$

A public probe q produces a normalized behavior signature

$$\sigma_i(p, q) = \mathcal{N}(\text{run}(q, R_i \oplus p)), \quad (4)$$

where normalization removes local path and formatting noise but preserves pass/fail outcomes, exceptions, assertions, API-visible values, and timeouts. A behavioral-disagreement certificate exists when one admissible public probe separates visible-passing candidates into at least two behavior classes:

$$\text{BDC}_i(q) = \mathbf{1}\{q \in Q_i, \text{goldblind}_i(q) = 1, \quad (5)$$

$$|\{\sigma_i(p, q) : p \in C_i\}| \geq 2\}.$$

The pass/fail subtype requires at least one passing and one failing signature among visible-passing candidates. After the audit index is frozen, the selected calibration candidate $s_i \in C_i$ is evaluated against held-out labels, and false acceptance is measured as

$$\text{FA}_i = \mathbf{1}[s_i \in C_i \wedge \exists h \in H_i : \sigma_i(s_i, h) = \text{fail}]. \quad (6)$$

The primary estimand is conditional on the selected certificate-producing path:

$$\theta_S = \Pr(\text{FA}_i = 1 \mid i \in S), \quad \hat{\theta}_S = \frac{1}{|S|} \sum_{i \in S} \text{FA}_i. \quad (7)$$

This formalization treats a certificate as evidence of unresolved public behavior; patch correctness is resolved only by later calibration or external judgment.

D. Candidate Generation and Visible Filtering

PatchCourt ingests candidate diffs from multiple lanes. The frozen path uses an interactive model-generated lane and a project-controlled batch lane; both keep candidate provenance so every certificate can be traced back to its source. The separate stress lane uses a locally served Qwen2.5-Coder model with structured search/replace generation. Table I records how each lane enters the claims.

Candidates are applied in isolated worktrees. Environment routing is per task because repository-level benchmarks include old Python runtimes, compiled extensions, generated version files, and package-specific test entry points. PatchCourt records apply status, visible status, execution-route metadata, compact outcome summaries, and candidate-source provenance.

E. Probe Layers

PatchCourt separates probe evidence into five layers: *source-isolated templates*, which are cheap discovery checks when full package execution is blocked; *issue-code probes*, which run executable public issue snippets; *full-package issue probes*, which import and drive the repository package around issue behavior; *package-integrity probes*, which test import or core API preservation; and *held-out calibration*, which runs evaluation-only labels after freeze.

This layering keeps each observation tied to the behavior it actually exercises. Package-integrity splits describe import or API regression risk; issue behavior is measured with issue-code, full-package issue probes, or held-out calibration. All-fail divergences are recorded separately from pass/fail behavioral certificates. Held-out failures enter only after the audit index is frozen. Post-disagreement overlays strengthen execution on already identified public behavior surfaces and are reported as selected upgrade evidence. Table II makes the probe provenance explicit, and Figure 4 gives compact code-facing examples for an issue-code probe and an independent-source stress probe.

F. Behavior Signatures and Certificates

Probe executions are normalized into behavior summaries: which candidate and public probe were checked, whether the

TABLE I
CANDIDATE-SOURCE PROVENANCE FOR PAPER-FACING CLAIMS.

Lane	Frozen	Candidate evidence	Use in claims
Interactive model-generated lane	7	Model patch evidence; thirteen certificates before conservative deduplication.	Part of the selected certificate-producing path.
Project-controlled batch lane	23	Additional calibration diversity with source provenance preserved in the artifact.	Main selected-path source, reported as project-controlled candidate evidence.
Open-source Qwen stress lane	0	Structured multi-sample edits yield visible-pass candidates and disagreement evidence.	Separate non-Codex stress test, reported outside the frozen 30.

<pre># SymPy-23117 issue-code reproducer from sympy import Array a = Array([]) print("array", list(a)) print("shape", a.shape) A/B/C: pass -> array []; shape (0,) D: fail -> empty-shape ValueError</pre>	<pre># Seaborn-3407 Qwen stress probe probe: pairplot MultiIndex reproducer qwen sample q0: pass -> axes_shape (4, 4) qwen samples q1..q4: fail -> import or indexing path evidence: one issue-specific Qwen BDC</pre>
--	--

Fig. 4. Representative code-facing public probes. Left: an issue-code SymPy reproducer separates three passing candidates from one failing candidate. Right: the strongest Qwen issue-specific stress case separates one visible-pass sample that completes the public Seaborn reproducer from four visible-pass samples that fail.

TABLE II
PROBE PROVENANCE AND TIMING. HELD-OUT LABELS ARE READ ONLY AFTER FREEZE.

Layer	Timing	Interpretation
Source-isolated	Post-issue and pre-held-out; often after candidate-context materialization, before calibration labels.	Cheap issue-facing discovery signal; it flags specification debt before package execution is available.
Issue-code	Public snippet exists pre-candidate; adaptation/routing may be post-disagreement, still gold-blind.	Issue-tied execution evidence for selected records.
Full-package	Mostly post-disagreement audit-upgrade overlay for selected targets, before label reading.	Robustness check for selected certificates under repository package execution.
Integrity	Post-Qwen visible-pass layer.	Regression/API-preservation evidence.
Held-out	After the frozen index is written.	Calibration label, never pre-freeze probe.

probe passed, failed, or timed out, and a compact behavioral fingerprint. A behavior-disagreement certificate is emitted when visible-passing candidates for the same task and probe produce at least two behavior groups. The strongest behavior-disagreement certificates have a pass/fail split on issue-related probes. All-fail behavioral divergences are diagnostic and reported separately.

G. Audit, Freeze, and Triage

PatchCourt freezes a conservative certificate index before reading held-out labels. The frozen set contains 30 task-level certificates. Certificate audit labels each record as retain, downgrade, or reject; the frozen pass yields 22 retain, 8 downgrade, and 0 reject. After freeze, PatchCourt runs held-out FAIL_TO_PASS calibration and records false acceptance. The audit saw probe layer and candidate source because those fields determine what each certificate can support; held-out outcomes remained hidden until the index was frozen. The result is a transparent single-pass audit; inter-rater reliability remains a validity threat. The artifact exposes stable certificate links, pass/fail behavior, audit rationales, downgrade reasons, and selected calibration candidates. The rubric is deliberately conservative: retain means that the probe is issue-relevant or strongly issue-adjacent, gold-blind, visible-pass-backed, and

behaviorally meaningful; downgrade means that the certificate remains useful but has a source-isolated, all-fail, routing, or missing-full-package caveat; reject means that the record is not issue-relevant, not reproducible, not gold-blind, or not backed by visible-pass candidates. The frozen input had already filtered to conservative pass/fail records, so no record in the frozen 30 met the reject criteria; weaker surviving cases were marked as downgrades. The single-pass audit is made inspectable through examples: retained Matplotlib and Seaborn records expose issue/full-package pass/fail behavior on visible-pass candidates, while downgraded Matplotlib and Sphinx records remain in the index with source-isolated, routing, or missing-upgrade caveats.

PatchCourt computes candidate-conditional oracle-gap lower bound (CC-OGB), a specification-debt indicator, audit-weighted risk, and abstention rules. CC-OGB is reported as a lower-bound risk signal. It summarizes unresolved behavior mass within the accepted candidate pool. Specification debt records reasons to keep a certificate under review, such as source-isolated status, downgraded audit, missing full-package validation, or all-fail divergence.

H. The Freeze-and-Calibrate Protocol

The empirical denominator is produced by this protocol before calibration. PatchCourt begins from a 50-task pilot and admits a paper-facing record when four conditions hold: at least one visible-pass candidate exists; a gold-blind public probe separates visible-pass candidates; the audit record can be traced to a task, probe, source lane, and route; and the selected calibration candidate can be executed after freeze. This yields 30 task-level records in the conservative certificate-producing path. The choice favors fully auditable rows over prevalence coverage: every row has a certificate, probe provenance, candidate-source provenance, audit status, route record, selected candidate, and held-out outcome. Wilson intervals are reported for $\hat{\theta}_S$, the selected-path estimand.

IV. RESEARCH QUESTIONS AND SETUP

RQ1. How often do visible/probe-accepted candidates become false acceptances under post-freeze held-out calibration?

RQ2. How much certificate evidence survives stronger issue-code or full-package probes?

RQ3 (triage calibration). Do pre-freeze signals provide calibrated triage, or only exploratory review cues?

RQ4 (independent-source stress test). What changes when candidate patches come from a locally served Qwen model?

A. Dataset and Freeze

The selected-path study uses a frozen 30-certificate audit set built from a 50-task SWE-bench-style pilot. The pilot was selected to stress repository-level repair conditions that matter for acceptance audit: visible commands, real package environments, issue text, candidate patches, and executable public probes. From that pilot, the paper-facing denominator keeps only conservative task-level records that pass the freeze-and-calibrate protocol described above. This design makes each row checkable enough for an ICSE artifact review and keeps the rate tied to the selected certificate-producing path.

The unit of analysis is one frozen task-level certificate record, not an individual model sample, repository family, or benchmark issue pool. Each record stores the task, public probe, selected candidate, source lane, evidence layer, audit status, routing summary, and post-freeze held-out outcome. When several candidate disagreements exist for a task, the audit index deduplicates to one conservative paper-facing record using a fixed precedence: pass/fail certificates over all-fail divergences, stronger public execution layers over weaker source-isolated layers when available at freeze time, retain over downgrade, and stable record order as the final tie-breaker. The held-out calibration candidate is not chosen after seeing held-out labels. It is the probe-passing visible-pass candidate selected by the frozen record, preferring model-generated patches over no-op controls and then the deterministic record order. All 30 stored calibration candidates pass their pre-freeze probe. The freeze point is the creation of the frozen certificate index; held-out calibration reads evaluation-only labels only afterward. Figure 5 shows the denominator, audit, and calibration splits that define the selected path. The anonymous artifact includes the full numeric funnel and a 50-task pilot drop-reason table that records which pilot tasks entered the frozen 30, which broad all-fail record was excluded, and which tasks did not produce conservative paper-facing certificates. The study therefore answers a conditional question: among auditable visible-pass cases that already show public behavioral disagreement, how often does acceptance fail when the held-out layer is finally opened?

Table III records where routing changed executability. The frozen certificate index itself did not change after labels were read; routing changes are append-only execution repairs or stronger-probe overlays.

B. Metrics and Statistical Treatment

We report held-out executability, held-out false-acceptance count and Wilson interval, specification-debt count, pass/fail BDC yield under upgraded probes, correlation and rerank

TABLE III
ROUTE-STABILITY CONTROLS. ROUTING REPAIRS IMPROVE EXECUTABILITY AND STRONGER-PROBE COVERAGE WITHOUT REWRITING THE FROZEN CERTIFICATE INDEX.

Layer	Before routing repair	Paper-facing use
Held-out FAIL_TO_PASS	15/30 executable	30/30 executable in Table V.
Main probe upgrades	2/10 upgraded BDC targets	6/10 pass/fail full-package or issue-code BDC targets.
Frozen audit index	30 conservative records	Unchanged source of truth for calibration.
Independent-source gate	Qwen stress lane only	Batch candidates remain project-controlled evidence.

TABLE IV
COST AND OVERHEAD SUMMARY FOR THE PAPER-FACING LAYERS. THE TABLE REPORTS WORK UNITS AND EXECUTION COUNTS.

Item	Value	Note
Candidate slots per pilot task	4.0	Three non-control slots plus one no-op control.
Candidate-probe executions	148	37 certificate records times four candidate versions.
Main upgrade probe runs	40	10 targets times four candidate versions.
Extra full-package probe runs	40	10 targets times four candidate versions.
Held-out calibration runs	30	Executed after freeze; all records executable.
Certificate-audit work units	30	Single structured audit pass; minutes not logged.
Route blockers before / after repair	15/30 to 0/30	Held-out unavailable records before and after routing.
Upgrade non-BDC targets before / after repair	8/10 to 4/10	Remaining failures are old-dependency, all-pass, or all-fail candidate-pool limits.

scores, abstention coverage, and independent-source applicability. All primary proportions use Wilson 95% intervals. For RQ3, random-review precision uses the hypergeometric baseline implied by 17 false acceptances among the 30 records in the selected certificate-producing path: expected precision is 0.567 at any fixed review budget, with central 95% random intervals of [0.30, 0.80] at $k = 10$ and [0.45, 0.70] at $k = 20$. Ranking and correlation results are reported as preliminary triage evidence.

V. RESULTS

Figure 6 grounds the aggregate results in one quantified certificate-level case before the RQ-specific tables. It shows the candidate-level public-probe split and the record-level held-out label without treating the certificate as a held-out label.

A. RQ1: Held-Out False Acceptance

Table V summarizes the frozen calibration within the selected certificate-producing path. All 30 frozen held-out calibrations execute. Of these selected records, 17 are false acceptances under held-out FAIL_TO_PASS labels. Specification debt is present in 26 of 30 cases. False acceptance remains common after certificate audit: retained records have 13/22 false acceptances and downgraded records have 4/8. Specification debt is a conservative high-coverage signal in

Selection and calibration denominators

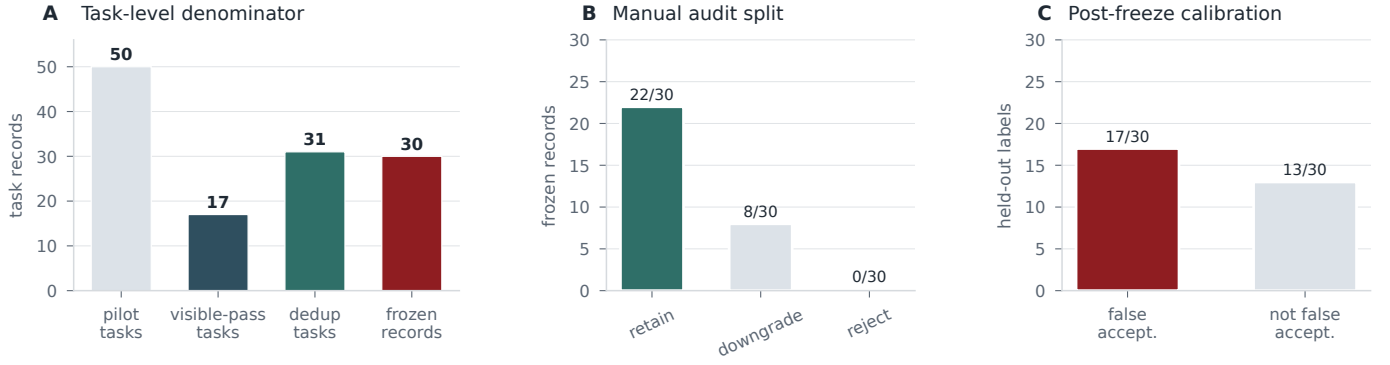


Fig. 5. Selection and calibration denominators for the selected certificate-producing path. The panels separate task-level selection counts, manual audit labels, and post-freeze held-out labels so the ‘17/30’ result remains tied to the frozen record set.

Matplotlib certificate: candidate-level behavior statistics

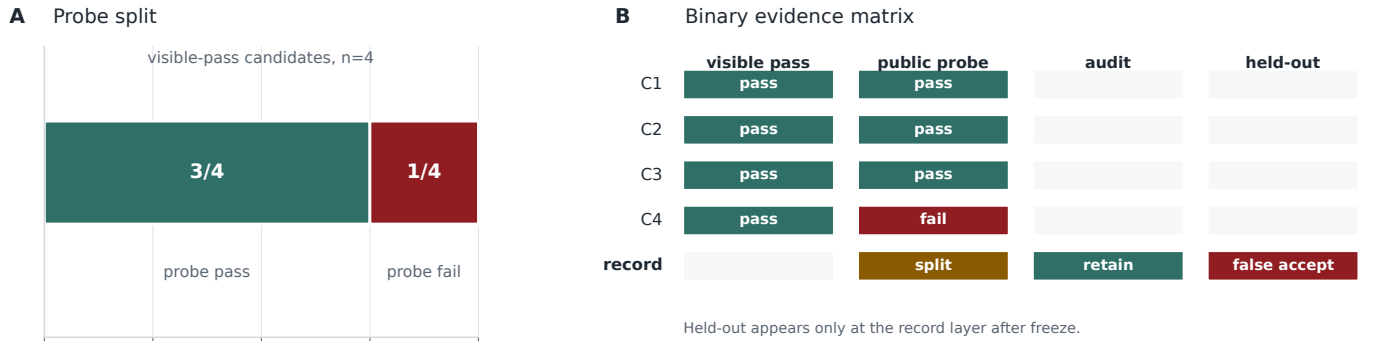


Fig. 6. Quantified certificate evidence for the Matplotlib all-NaN bar-plot case. The public probe splits four visible-passing candidates into three probe-pass and one probe-fail candidates; the record is retained before freeze and receives its false-acceptance label only after held-out calibration.

TABLE V
FROZEN HELD-OUT CALIBRATION AFTER THE CERTIFICATE INDEX IS FROZEN.

Quantity	Value
Frozen conservative certificates	30
Certificate audit retain/downgrade/reject	22/8/0
Held-out executable tasks	30/30
Selected-path false acceptances	17/30
Wilson 95% CI for false acceptance	[0.392, 0.726]
Specification-debt signal	26/30
Wilson 95% CI for specification debt	[0.703, 0.947]
Retain false-acceptance rate	13/22
Downgrade false-acceptance rate	4/8
Spec-debt-present false-acceptance rate	17/26
Spec-debt-absent false-acceptance rate	0/4

this set: all 17 false acceptances are in the specification-debt-present stratum, while the four debt-absent records have zero false acceptances.

Finding 1. Table V shows that visible/probe acceptance leaves substantial residual risk in the selected frozen audit set. On this path, visible/probe acceptance is a risky endpoint unless the residual public behavior is inspected.

B. RQ2: Stronger Probe Validation

Table VI reports both probe-upgrade layers. The upgrade layer executes 40 probes across 10 high-signal targets. Six targets yield pass/fail behavior-disagreement certificates under full-package or issue-code execution: Django empty sitemap last-modified, Django template join autoescape, Matplotlib all-NaN bar plot, Seaborn MultiIndex pairplot, Scikit-Learn LabelEncoder empty transform, and SymPy empty array. An additional full-package layer over 10 targets yields seven BDC tasks, six of which are pass/fail and one of which is all-fail divergence.

Finding 2. Table VI shows that selected certificates can survive issue-code and full-package upgrades, extending the evidence beyond source-isolated templates. The upgraded subset is small, so pass/fail BDCs and all-fail divergences are reported as separate evidence types.

C. RQ3 Triage Signals

Table VII shows that raw CC-OGB alone is under-calibrated in the frozen 30-task table. Specification debt and audit-weighted risk provide modest review-budget signals: audit-weighted risk increases precision@10 from 0.400 to 0.600 and precision@20 from 0.550 to 0.650 relative to raw CC-OGB. These values remain inside the finite-sample random intervals

TABLE VI

PROBE-UPGRADE EVIDENCE. STRONGER ISSUE-CODE AND FULL-PACKAGE PROBES PRESERVE PASS/FAIL DISAGREEMENT FOR SELECTED TARGETS; ALL-FAIL DIVERGENCE IS REPORTED SEPARATELY.

Layer	Targets	Pass/fail BDCs	Positive tasks / notes
Main full-package or issue-code upgrades	10	6	Django-16255, Django-16873, Matplotlib-24149, Seaborn-3407, Scikit-Learn-10508, and SymPy-23117. Wilson 95% CI: [0.313, 0.832].
Extra full-package upgrades	10	6	Seven total BDC tasks; six pass/fail and one all-fail divergence. This layer strengthens package-level robustness without being merged into the main issue-specific count.
Qwen issue/full-package probes	11 visible-pass tasks	1	Seaborn-3407 is the only Qwen issue-specific pass/fail BDC after multi-sample generation and rescue.
Qwen package-integrity probes	11 visible-pass tasks	4	Integrity positives for Seaborn, Flask, and Scikit-Learn expose full-package regression risk.

TABLE VII

TRIAGE AND ABSTENTION. RANDOM REVIEW HAS EXPECTED $P@10/P@20$ OF 0.567/0.567; CENTRAL 95% RANDOM INTERVALS ARE [0.30, 0.80] AT $k = 10$ AND [0.45, 0.70] AT $k = 20$.

Signal	Pearson	AP	$P@10/P@20$
Raw CC-OGB	-0.116	0.557	0.400 / 0.550
CC-OGB + spec debt	0.146	0.612	0.500 / 0.650
Audit-weighted risk	0.129	0.617	0.600 / 0.650
Spec-debt indicator	0.449	0.612	—
Random review order	—	—	0.567 / 0.567
Abstention rule	Rate	Capture	Residual FA
CC-OGB ≥ 0.50	0.167	0.118	0.600
CC-OGB ≥ 0.75	0.067	0.059	0.571
Specification debt	0.867	1.000	0.000

around random review, so the signal is best read as a review cue at this sample size. The specification-debt abstention rule is conservative and high coverage: it abstains on 26/30 tasks and captures all 17 false acceptances in the selected certificate-producing path. The rule is broad, but it shows that the ledger can drive explicit risk-control decisions.

Finding 3. Table VII reads as a small-sample calibration result. Specification debt is conservative and high coverage, while the broad abstention rate and random-order intervals keep raw disagreement scores in the category of review cues at this sample size.

D. Independent Qwen Stress Test

The independent-source experiment asks how far the evidence pipeline carries when candidate patches come from a non-Codex local model source.

Naive diff generation with Qwen rarely reaches the apply-and-visible-pass gate in a form that can yield certificate evidence. Structured search/replace generation improves applicability and creates a visible-pass stress pool. On the frozen set, structured generation yields applicable candidates for 17 tasks and visible-pass candidates for 11 tasks. Multi-sample generation over those visible-pass tasks produces a larger candidate pool, but issue-specific pass/fail disagreement remains rare.

Table VIII summarizes the supported inference for each evidence type. This lane is a partial-positive stress test: Patch-Court ingests a real non-Codex source and finds nonzero disagreement, while the observed issue-specific yield remains

low. Figure 4 shows the strongest issue-specific Qwen case as a code-facing public probe and normalized output split.

Qwen stress lane. Structured generation turns an applicability failure into a visible-pass multi-sample pool. The retained evidence consists of one issue-specific pass/fail certificate and four package-integrity certificates. Raw generation and applicability counts are preserved in the artifact as diagnostic evidence.

Figure 7 consolidates the main result denominators after the RQ-specific tables. Panel A shows the selected-path calibration rates and Wilson intervals. Panel B separates pass/fail upgrade yield from other disagreement and no-BDC outcomes. Panel C records the routing repairs that made calibration and upgrade execution complete enough for the paper-facing tables. Panel D shows the reranking result: the best review cue improves over raw CC-OGB, but the finite-sample random baseline remains close. The artifact contains the corresponding single-purpose plots for calibration rates, probe-upgrade yield, source/layer mix, route stability, rerank/abstain, Qwen yield, and repository-level outcome distribution.

VI. QUALITATIVE CASES

The following cases put the aggregate tables back into certificate-level behavior. Each case links a table-level result to an inspectable audit row. Code-facing evidence appears in Figures 2, 4, and 8.

A. Seaborn MultiIndex Pairplot

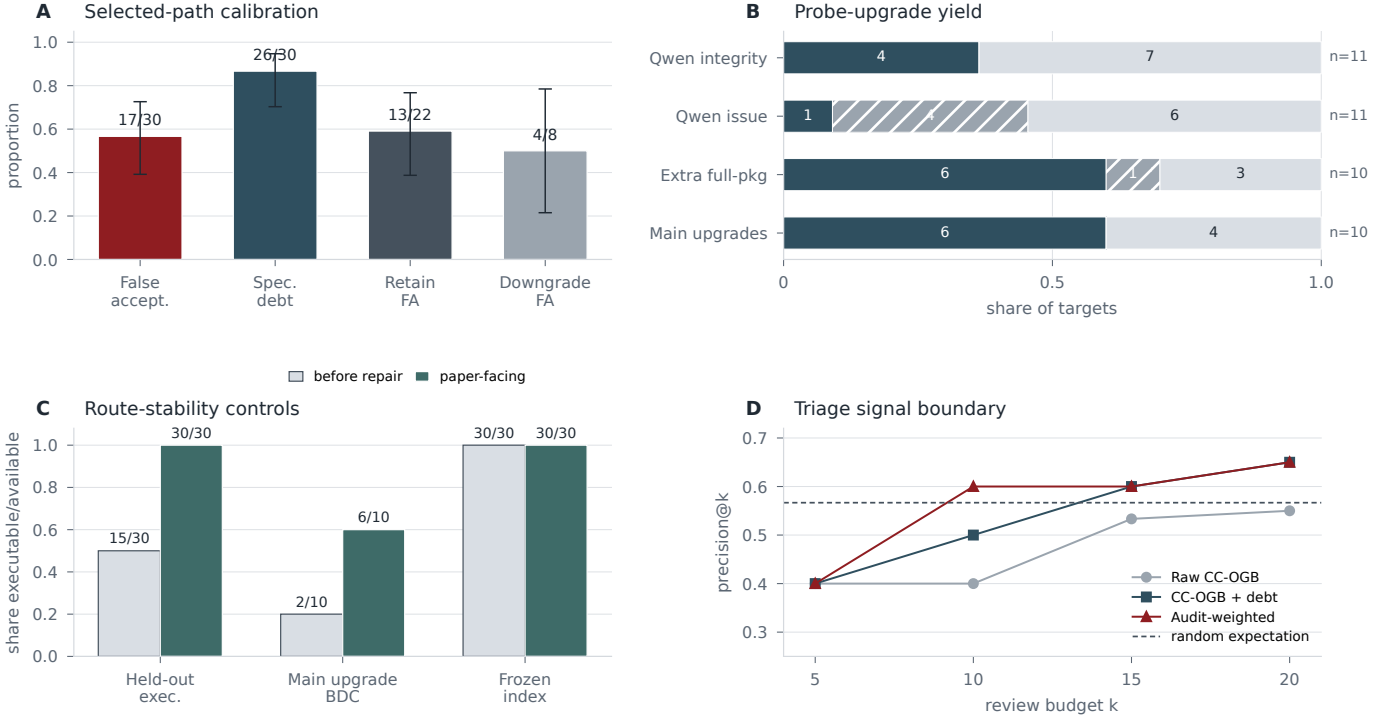
Seaborn-3407 is the strongest cross-source case. It appears in the main upgraded probe layer and is also the only Qwen issue/full-package pass/fail BDC after multi-sample generation. The issue-level package probe separates visible-passing edits: the main lane reports three pass and one fail, and the Qwen lane reports one pass and four fail, with the passing group constructing the 4-by-4 axes grid in Figure 4. The certificate supports independent-source compatibility without carrying a broad Qwen reproduction claim.

B. Matplotlib All-NaN Bar Plot

Matplotlib-24149 links a full-package upgraded probe to a post-freeze held-out false acceptance. Visible tests accept candidates that differ on an all-NaN plotting edge case. After routing repair, the package-level probe records three pass, one fail, and no timeouts: the passing group preserves one bar with

TABLE VIII
EVIDENCE READING BY RESULT TYPE.

Evidence type	Supported inference	Reading condition
Frozen held-out calibration (17/30 false acceptance; 26/30 spec debt)	Visible/probe acceptance often fails held-out labels in the selected certificate-producing path.	Conditional on the frozen selected path.
Main and extra probe upgrades (6/10 and 6/10 pass/fail)	Stronger full-package or issue-code probes preserve disagreement for selected targets.	Evidence for selected upgrade targets.
Qwen issue-specific pass/fail BDC (1 task)	A single independent-source issue-specific compatibility case; strongest case is Seaborn MultiIndex pairplot.	Low observed issue-specific yield.
Qwen package-integrity BDCs (4 tasks)	Independent-source visible-pass patches can break or preserve full-package behavior differently.	Integrity/regression reading.
Qwen all-fail divergences and rescue negatives	Diagnostic behavior diversity and a documented negative result.	Negative applicability evidence.



All rates are conditional on the selected certificate-producing path; Qwen panels summarize stress-test evidence, not a broad external-model baseline.

Fig. 7. Quantitative result summary. Panel A reports selected-path calibration rates with Wilson 95% intervals. Panel B reports pass/fail BDC yield by probe layer and keeps Qwen issue and integrity evidence separated. Panel C shows routing repairs without changing the frozen audit index. Panel D summarizes rerank precision against the random-review reference.

a NaN coordinate, while the failing group exits through the Matplotlib axes stack. The certificate localizes the behavioral split; the false-acceptance label comes only from later frozen held-out calibration.

C. Scikit-Learn LabelEncoder Empty Transform

Scikit-Learn-10508 separates issue behavior from package-integrity risk. In the main lane, the LabelEncoder empty-transform reproducer yields a pass/fail certificate once compiled-extension/build routing is repaired. The record has three pass, one fail, and no timeouts: the passing group preserves an empty one-dimensional output, while the failing group hits a NumPy casting path. In the Qwen lane, issue-specific probes remain all-fail, while a package-integrity probe

reports two pass and three fail on whether LabelEncoder can be imported and constructed; PatchCourt records that split as package-risk evidence.

D. Django Semantic Split

Django-16873 is a full-package semantic split under template rendering with autoscape disabled. The record has three pass, one fail, and no timeouts: the passing group preserves literal line-break separators, while the failing group escapes them and raises an assertion. The later held-out label is not false acceptance, so the case shows how behavior-disagreement certificates flag unresolved semantics without becoming hidden-test labels.

```
# Django-16873 full-package public probe
out = render_join(autoescape=False)

A/B/C: pass -> "alpha\nbeta"
D:      fail -> escaped separator assertion

certificate: pass/fail BDC before freeze
evidence: semantic split before held-out labels
```

Fig. 8. Code-facing Django case probe. The public full-package probe splits three visible-passing candidates that preserve separator behavior from one that escapes it.

E. SymPy Issue-Code Layer

SymPy task 23117 demonstrates the issue-code layer. The public snippet in Figure 4 yields three pass, one fail, and no timeouts, with the passing group preserving an empty array result. This issue-tied layer is stronger than a source-isolated template while remaining separate from held-out calibration.

VII. RELATED WORK

A. Repository-Level Repair, Agents, and Benchmarks

Repository-level repair sits on benchmark and model infrastructure. Defects4J, ManyBugs, Bugs.jar, Bears, BugsInPy, and QuixBugs established controlled real-bug substrates for repair and testing research [8]–[13]. SWE-bench and SWE-bench+ move the unit of work to GitHub issues and real repository snapshots [1], [14]. Code models and code benchmarks provide the upstream capability base for these systems [15]–[23]. Agent work studies repository search, tool use, turn management, and candidate-fix generation [2], [3], [24]–[27]. PatchCourt enters after that generation process. It audits the moment when a visible-pass candidate would otherwise be accepted and records what public evidence remains unresolved.

B. LLM Repair and APR Overfitting

Automated program repair has long separated test-passing plausibility from semantic correctness, across generate-and-validate, template, semantic, and learning-based repair [28]–[40]. Patch plausibility and overfitting studies show why passing tests can be insufficient [4]–[6], [41]. Recent ICSE work revisits APR in the LLM era through code-language-model impact, bug-report/test coupling, objective alignment, input reduction, and unnaturalness-based ranking [42]–[46]. PatchCourt operationalizes a narrower acceptance-time question for repository-level LLM patch pools: after visible tests pass, can public, gold-blind behavior measurements expose an auditable risk record before benchmark labels are opened?

C. Patch Correctness, Benchmark Validity, and Contamination

Recent SWE-bench correctness work asks whether benchmark-accepted issues are genuinely fixed and uses PatchDiff to compare generated patches against human-written ground-truth patches [7]. Diagnostic benchmark work studies whether strong SWE-bench performance may reflect memorization or benchmark exposure rather than issue reasoning [47], [48]. These studies challenge the meaning of a benchmark pass after evaluation. PatchCourt takes a prospective route: it freezes an evidence ledger before developer patches, hidden tests, or stronger labels enter the process, then uses held-out calibration to measure alignment

with later failure. The payoff is an acceptance-time signal that can be inspected before benchmark answers are opened.

D. Generated Tests, Oracles, and Validation

Generated tests can strengthen repair validation, but they face the test oracle problem: producing an input is easier than knowing the expected behavior. Classic testing work developed feedback-directed random testing, symbolic execution, dynamic test generation, and mutation-analysis foundations for exposing untested behavior [49]–[54]. Search-based and LLM-based testing studies continue this line with generated tests, bug reproduction, and execution feedback [55]–[61]. Oracle-generation work, including TOGA, neural-oracle evaluations, and TOGLL, studies when automated or LLM-produced assertions become correct and strong enough for testing [62]–[66]. PatchCourt treats probes as evidence-producing instruments whose strength depends on provenance and execution layer. Their evidentiary weight depends on the layer: source-isolated, issue-code, full-package issue, package-integrity, or post-freeze held-out calibration.

VIII. THREATS TO VALIDITY

Internal validity. Probe code may encode an incomplete interpretation of the issue. Environment routing may introduce artifacts, especially for repositories with compiled extensions, old dependencies, or package-specific entry points. PatchCourt mitigates this by logging route metadata, reporting route stability, and separating source-isolated, issue-code, full-package, package-integrity, and held-out layers. The rubric still relies on human judgment; certificate links, pass/fail behavior, audit labels, and downgrade reasons are exposed in the artifact.

Construct validity. behavior-disagreement certificates localize public behavioral disagreement, while CC-OGB summarizes a candidate-conditional lower-bound signal. All-fail divergence is weaker than pass/fail disagreement, and package-integrity probes measure regression or API preservation; the tables keep those evidence types separate from issue behavior.

External validity. The main calibration set contains 30 frozen certificates selected through an audit pipeline. The resulting rates describe this selected certificate-producing path. Broader repository, language, and candidate-source coverage is needed before estimating field-wide behavior. The selection funnel in Figure 5 is included to make this selected-set boundary explicit.

Conclusion validity. The correlation, rerank, and abstain results are small- N and preliminary. Specification debt performs strongly in this table, but the pattern may reflect the selected certificate set. The triage results are an early review-prioritization signal, with predictor validation left to larger sets.

Independent-source validity. One issue-specific pass/fail BDC and four package-integrity BDCs make the Qwen lane a stress test with low issue-specific yield. The direct-diff Qwen run remains a negative baseline, and the project-controlled batch lane remains separate from an authenticated external-model baseline.

Gold-boundary validity. PatchCourt’s main design invariant is the pre-freeze/post-freeze separation. The artifact must keep this separation inspectable. Any future experiment that consults developer patches, test patches, FAIL_TO_PASS, or PASS_TO_PASS before freeze should be reported as a separate retrospective analysis.

IX. ARTIFACT

The anonymous artifact is designed for inspect-only reproduction first, with optional deeper replay where compute and model access are available. At Level 0, reviewers inspect frozen tables, reports, certificates, and audit labels without external model access. At Level 1, they run the packaged verifier to regenerate paper-facing summaries from the shipped CSV, JSONL, and Markdown files. At Levels 2–3, they replay selected probes or regenerate candidates, visible tests, probes, BDCs, ledgers, and calibration with suitable runtimes and model access.

The bundle contains paper tables, result data files, frozen audit indices, Qwen audit reports, figure sources, rendered figures, and an anonymous claim-boundary card. It excludes passwords, API keys, transient cookies, private keys, and author-identifying machine paths. Execution identifiers remain inside index files for auditability, while the paper-facing text uses stable concepts.

X. DISCUSSION AND CONCLUSION

This paper studied a concrete failure mode in repository-level LLM repair: visible tests can accept patches that still encode unresolved issue behavior. We characterize that gap as an acceptance-time measurement problem. PatchCourt turns visible-passing patches into executable hypotheses, records public behavior disagreements before benchmark labels are read, and calibrates the frozen records after held-out labels become available.

The method follows a deliberately narrow flow. Candidate patches first pass through repository apply and visible-test filters. Public probes measure behavior at source-isolated, issue-code, full-package, and package-integrity layers. Executions are normalized into behavior signatures and retained as certificate records when the audit rubric finds actionable disagreement. After freeze, held-out calibration estimates how often the selected certificate-producing path corresponds to false acceptance or specification debt.

The main empirical result is that visible-pass acceptance is a poor stopping point on this selected path. Among 30 frozen certificate-producing records, 17 become false acceptances under post-freeze held-out calibration and 26 retain specification-debt signals. Stronger issue-code or full-package upgrades preserve pass/fail disagreement in 6/10 selected upgrades, and a separate package-level upgrade set preserves another 6/10 certificates. The Qwen stress lane demonstrates source compatibility with one issue-specific and four package-integrity disagreement cases, while keeping the main claim anchored in the audited frozen path.

The broader implication is operational. When public executable evidence still shows unresolved behavior, visible tests should trigger inspection, reranking, or abstention before a repository-level repair is accepted. Future work should expand the selected path across more repositories and model sources, add multi-rater audit, and integrate certificate evidence into repair agents before deployment-time acceptance decisions.

REFERENCES

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world github issues?” in *Proceedings of the International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VTF8yNQM66>
- [2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: https://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
- [3] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “AutoCodeRover: Autonomous program improvement,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1592–1604. [Online]. Available: <https://doi.org/10.1145/3650212.3680384>
- [4] Z. Qi, F. Long, S. Achour, and M. C. Rinard, “An analysis of patch plausibility and correctness for generate-and-validate patch generation systems,” in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, 2015, pp. 24–36. [Online]. Available: <https://doi.org/10.1145/2771783.2771791>
- [5] E. K. Smith, E. T. Barr, C. L. Goues, and Y. Brun, “Is the cure worse than the disease? overfitting in automated program repair,” in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 532–543. [Online]. Available: <https://doi.org/10.1145/2786805.2786825>
- [6] H. Ye, M. Martinez, and M. Monperrus, “Automated patch assessment for program repair at scale,” *Empirical Software Engineering*, vol. 26, no. 20, 2021. [Online]. Available: <https://doi.org/10.1007/s10664-020-09920-w>
- [7] Y. Wang, M. Pradel, and Z. Liu, “Are ‘solved issues’ in SWE-bench really solved correctly? an empirical study,” in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, 2026. [Online]. Available: <https://doi.org/10.1145/3744916.3764576>
- [8] R. Just, D. Jalali, and M. D. Ernst, “Defects4J: A database of existing faults to enable controlled testing studies for java programs,” in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 2014, pp. 437–440. [Online]. Available: <https://doi.org/10.1145/2610384.2628055>
- [9] C. L. Goues, N. Holschulte, E. K. Smith, Y. Brun, P. Devanbu, S. Forrest, and W. Weimer, “The ManyBugs and IntroClass benchmarks for automated repair of C programs,” in *Proceedings of the 37th International Conference on Software Engineering*, 2015, pp. 396–407. [Online]. Available: <https://doi.org/10.1109/ICSE.2015.155>
- [10] R. K. Saha, Y. Lyu, H. Yoshida, and M. R. Prasad, “Bugs.jar: A large-scale, diverse dataset of real-world java bugs,” in *Proceedings of the 15th International Conference on Mining Software Repositories*, 2018, pp. 10–13. [Online]. Available: <https://doi.org/10.1145/3196398.3196473>
- [11] F. Madeiral, S. Urli, M. Maia, and M. Monperrus, “Bears: An extensible java bug benchmark for automatic program repair studies,” in *Proceedings of the 26th IEEE International Conference on Software Analysis, Evolution and Reengineering*, 2019, pp. 468–478. [Online]. Available: <https://doi.org/10.1109/SANER.2019.8667991>
- [12] R. Widayarsi, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, and E. L. Ouh, “BugsInPy: A database of existing bugs in python programs to enable controlled testing and debugging studies,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 1556–1560. [Online]. Available: <https://doi.org/10.1145/3368089.3417943>

- [13] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama, “QuixBugs: A multi-lingual program repair benchmark set based on the quixey challenge,” in *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, 2017, pp. 55–56. [Online]. Available: <https://doi.org/10.1145/3135932.3135941>
- [14] R. Aleithan, H. Xue, M. M. Mohajer, E. Nnorom, G. Uddin, and S. Wang, “SWE-Bench+: Enhanced coding benchmark for LLMs,” in *arXiv preprint arXiv:2410.06992*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.06992>
- [15] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547. [Online]. Available: <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
- [16] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. B. Clement, D. Drain, D. Jiang, D. Tang *et al.*, “CodeXGLUE: A machine learning benchmark dataset for code understanding and generation,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. [Online]. Available: <https://openreview.net/forum?id=6lE4dQXaUcb>
- [17] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” in *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [18] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, “Program synthesis with large language models,” in *arXiv preprint arXiv:2108.07732*, 2021. [Online]. Available: <https://arxiv.org/abs/2108.07732>
- [19] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, “CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 8696–8708. [Online]. Available: <https://doi.org/10.18653/v1/2021.emnlp-main.685>
- [20] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago, T. Hubert, P. Choy, C. de Masson d’Autume, I. Babuschkin, X. Chen *et al.*, “Competition-level code generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022. [Online]. Available: <https://doi.org/10.1126/science.abq1158>
- [21] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, “CodeGen: An open large language model for code with multi-turn program synthesis,” in *Proceedings of the 11th International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=iaYcJKpY2B_
- [22] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, “Code Llama: Open foundation models for code,” in *arXiv preprint arXiv:2308.12950*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.12950>
- [23] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li *et al.*, “DeepSeek-Coder: When the large language model meets programming,” in *arXiv preprint arXiv:2401.14196*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.14196>
- [24] I. Bouzenia, P. Devanbu, and M. Pradel, “RepairAgent: An autonomous, LLM-based agent for program repair,” in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, 2025, pp. 2188–2200. [Online]. Available: <https://doi.org/10.1109/ICSE55347.2025.00157>
- [25] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying LLM-based software engineering agents,” in *arXiv preprint arXiv:2407.01489*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.01489>
- [26] P. Gao and C. Peng, “More with less: An empirical study of turn-control strategies for efficient coding agents,” in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. [Online]. Available: <https://doi.org/10.1145/3744916.3773250>
- [27] Z. Tian, P. Gao, J. Chen, and C. Peng, “Agent-based ensemble reasoning for repository-level issue resolution,” in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. [Online]. Available: <https://raw.githubusercontent.com/tianzhaotju/preprints/main/ICSE%26%20-%20Agent-Based%20Ensemble%20Reasoning%20for%20Repository-Level%20Issue%20Resolution.pdf>
- [28] W. Weimer, T. Nguyen, C. L. Goues, and S. Forrest, “Automatically finding patches using genetic programming,” in *Proceedings of the 31st International Conference on Software Engineering*, 2009, pp. 364–374. [Online]. Available: <https://doi.org/10.1109/ICSE.2009.5070536>
- [29] C. L. Goues, M. Dewey-Vogt, S. Forrest, and W. Weimer, “A systematic study of automated program repair: Fixing 55 out of 105 bugs for \$8 each,” in *Proceedings of the 34th International Conference on Software Engineering*, 2012, pp. 3–13. [Online]. Available: <https://doi.org/10.1109/ICSE.2012.6227211>
- [30] D. Kim, J. Nam, J. Song, and S. Kim, “Automatic patch generation learned from human-written patches,” in *Proceedings of the 35th International Conference on Software Engineering*, 2013, pp. 802–811. [Online]. Available: <https://doi.org/10.1109/ICSE.2013.6606626>
- [31] H. D. T. Nguyen, D. Qi, A. Roychoudhury, and S. Chandra, “SemFix: Program repair via semantic analysis,” in *Proceedings of the 35th International Conference on Software Engineering*, 2013, pp. 772–781. [Online]. Available: <https://doi.org/10.1109/ICSE.2013.6606623>
- [32] F. Long and M. Rinard, “Automatic patch generation by learning correct code,” in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2016, pp. 298–312. [Online]. Available: <https://doi.org/10.1145/2884781.2884807>
- [33] S. Mechtaev, J. Yi, and A. Roychoudhury, “Angelix: Scalable multiline program patch synthesis via symbolic analysis,” in *Proceedings of the 38th International Conference on Software Engineering*, 2016, pp. 691–701. [Online]. Available: <https://doi.org/10.1145/2884781.2884807>
- [34] J. Xuan, M. Martinez, F. Demarco, M. Clément, S. L. Marcote, T. Durieux, D. L. Berre, and M. Monperrus, “Nopol: Automatic repair of conditional statement bugs in java programs,” *IEEE Transactions on Software Engineering*, vol. 43, no. 1, pp. 34–55, 2017. [Online]. Available: <https://doi.org/10.1109/TSE.2016.2560811>
- [35] M. Tufano, C. Watson, G. Bavota, M. D. Penta, M. White, and D. Poshyvanyk, “An empirical investigation into learning bug-fixing patches in the wild via neural machine translation,” in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 832–837. [Online]. Available: <https://doi.org/10.1145/3238147.3240732>
- [36] T. Lutellier, H. V. Pham, L. Pang, Y. Li, M. Wei, and L. Tan, “CoCoNuT: Combining context-aware neural translation models using ensemble for program repair,” in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 101–114. [Online]. Available: <https://doi.org/10.1145/3395363.3397369>
- [37] N. Jiang, T. Lutellier, and L. Tan, “CURE: Code-aware neural machine translation for automatic program repair,” in *Proceedings of the 43rd International Conference on Software Engineering*, 2021, pp. 1161–1173. [Online]. Available: <https://doi.org/10.1109/ICSE43902.2021.00107>
- [38] Q. Zhu, Z. Sun, Y. Xu, W. Hu, J. Chen, L. Bo, Q. Zheng, and T. Zhao, “A syntax-guided edit decoder for neural program repair,” in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 341–353. [Online]. Available: <https://doi.org/10.1145/3468264.3468544>
- [39] C. S. Xia and L. Zhang, “Less training, more repairing please: Revisiting automated program repair via zero-shot learning,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 959–971. [Online]. Available: <https://doi.org/10.1145/3540250.3549101>
- [40] C. S. Xia, Y. Wei, and L. Zhang, “Automated program repair in the era of large pre-trained language models,” in *Proceedings of the 45th International Conference on Software Engineering*, 2023, pp. 1482–1494. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00129>
- [41] L. Gazzola, D. Micucci, and L. Mariani, “Automatic software repair: A survey,” *IEEE Transactions on Software Engineering*, vol. 45, no. 1, pp. 34–67, 2019. [Online]. Available: <https://doi.org/10.1109/TSE.2017.2755013>
- [42] N. Jiang, K. Liu, T. Lutellier, and L. Tan, “Impact of code language models on automated program repair,” in *Proceedings of the 45th International Conference on Software Engineering*, 2023, pp. 1430–1442. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00125>
- [43] M. Motwani, M. D. Ernst, and Y. Brun, “Better automatic program repair by using bug reports and tests together,” in *Proceedings of the 45th International Conference on Software Engineering*, 2023. [Online].

Available: <https://people.cs.umass.edu/~brun/pubs/pubs/Motwani23icse.pdf>

- [44] J. Xu, Y. Fu, S. H. Tan, and P. He, "Aligning the objective of LLM-based program repair," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, 2025. [Online]. Available: <https://arxiv.org/pdf/2404.08877>
- [45] B. Yang, L. Ren, X. Yin, J. Ren, H. Tian, and S. Jin, "Input reduction enhanced LLM-based program repair," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. [Online]. Available: <https://arxiv.org/pdf/2507.15251>
- [46] A. Z. H. Yang, S. Kolak, V. J. Hellendoorn, R. Martins, and C. L. Goues, "Revisiting unnaturalness for automated program repair in the era of large language models," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, 2025. [Online]. Available: <https://arxiv.org/abs/2404.15236>
- [47] S. Liang, S. Garg, and R. Z. Moghaddam, "The SWE-Bench illusion: When state-of-the-art LLMs remember instead of reason," in *arXiv preprint arXiv:2506.12286*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.12286>
- [48] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and contamination free evaluation of large language models for code," in *Proceedings of the 13th International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=chfJJYC3iL>
- [49] P. Godefroid, N. Klarlund, and K. Sen, "DART: Directed automated random testing," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2005, pp. 213–223. [Online]. Available: <https://doi.org/10.1145/1065010.1065036>
- [50] K. Sen, D. Marinov, and G. Agha, "CUTE: A concolic unit testing engine for C," in *Proceedings of the 10th European Software Engineering Conference Held Jointly with the 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2005, pp. 263–272. [Online]. Available: <https://doi.org/10.1145/1081706.1081750>
- [51] C. Cadar, D. Dunbar, and D. Engler, "KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation*, 2008, pp. 209–224. [Online]. Available: https://www.usenix.org/legacy/event/osdi08/tech/full_papers/cadar/cadar.pdf
- [52] C. Pacheco, S. K. Lahiri, M. D. Ernst, and T. Ball, "Feedback-directed random test generation," in *Proceedings of the 29th International Conference on Software Engineering*, 2007, pp. 75–84. [Online]. Available: <https://doi.org/10.1109/ICSE.2007.37>
- [53] G. Fraser and A. Arcuri, "EvoSuite: Automatic test suite generation for object-oriented software," in *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*, 2011, pp. 416–419. [Online]. Available: <https://doi.org/10.1145/2025113.2025179>
- [54] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011. [Online]. Available: <https://doi.org/10.1109/TSE.2010.62>
- [55] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proceedings of the 45th International Conference on Software Engineering*, 2023, pp. 919–931. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00085>
- [56] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring LLM-based general bug reproduction," in *Proceedings of the 45th International Conference on Software Engineering*, 2023, pp. 2312–2323. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00194>
- [57] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "CodeT: Code generation with generated tests," in *Proceedings of the International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=ktw68Cmu9c>
- [58] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: <https://openreview.net/forum?id=1qv610Cu7>
- [59] T. Ahmed, J. Ganhotra, A. Shinnar, and M. Hirzel, "Heterogeneous prompting and execution feedback for SWE issue test generation and selection," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. [Online]. Available: <https://arxiv.org/pdf/2508.06365>
- [60] J. Shin, N. S. Harzevili, R. Aleithan, H. Hemmati, and S. Wang, "Retrieval-augmented test generation: How far are we?" in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. [Online]. Available: <https://arxiv.org/pdf/2409.12682>
- [61] A. Li, M. Liu, Z. Chen, Z. Pei, Z. Li, D. Dai, Y. Wang, and Z. Zheng, "Knowledge matters: Injecting project and testing knowledge into LLM-based unit test generation," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. [Online]. Available: <https://arxiv.org/pdf/2511.14224>
- [62] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015. [Online]. Available: <https://discovery.ucl.ac.uk/id/eprint/1471263>
- [63] M. Pezzè and C. Zhang, "Automated test oracles: A survey," in *Advances in Computers*. Elsevier, 2014, vol. 95, pp. 1–48. [Online]. Available: <https://doi.org/10.1016/B978-0-12-800160-8.00001-2>
- [64] E. Dinella, G. Ryan, T. Mytkowicz, and S. K. Lahiri, "TOGA: A neural method for test oracle generation," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2130–2141. [Online]. Available: <https://doi.org/10.1145/3510003.3510141>
- [65] S. B. Hossain, A. Filieri, M. B. Dwyer, and W. Visser, "Neural-based test oracle generation: A large-scale evaluation and lessons learned," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 120–132. [Online]. Available: <https://doi.org/10.1145/3611643.3616265>
- [66] S. B. Hossain and M. B. Dwyer, "TOGLL: Correct and strong test oracle generation with LLMs," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, 2025, pp. 1475–1487. [Online]. Available: <https://doi.org/10.1109/ICSE55347.2025.00098>